РК1

по курсу

«Парадигмы и конструкции языков программирования»

Выполнил студент группы ИУ5-33Б Ицхакина Валентина

2026 г.

Вариант 1 В
Код программы:

```python
class Student:  4 usages  Valentinaltshakina *
    """Студент (сторона «много»)"""
    def __init__(self, id, fio, score, group_id):  Valentinaltshakina *
        self.id = id
        self.fio = fio
        self.score = score  # Количественный признак (например, средний балл)
        self.group_id = group_id  # ID группы для связи один-ко-многим

class Group:  3 usages  Valentinaltshakina *
    """Группа (сторона «один»)"""
    def __init__(self, id, name):  Valentinaltshakina
        self.id = id
        self.name = name


class StudentGroup:  5 usages  new *
    """Связь многие-ко-многим между студентами и группами"""
    def __init__(self, group_id, student_id):  new *
        self.group_id = group_id
        self.student_id = student_id
# Тестовые данные: Группы
groups = [
    Group(1, 'ИУ5-21Б'),
    Group(2, 'ИУ6-22Б'),
    Group(3, 'ИУ7-23Б'),
]
# Тестовые данные: Студенты (с указанием количественного признака score и фамилиями на «А» для запроса №1
students = [
    Student(1, 'Аристархов', 4.5, 1),
    Student(2, 'Петров', 3.8, 2),
    Student(3, 'Андреев', 4.9, 1),
    Student(4, 'Иванов', 4.0, 3),
]
# Тестовые данные: Связь многие-ко-многим
students_groups = [
    StudentGroup(1, 1),
    StudentGroup(1, 3),
    StudentGroup(2, 2),
    StudentGroup(3, 4),
    StudentGroup(2, 1),
]
```

```python
def main():  1 usage  Valentinaltshakina *
    # 1. Соединение данных один-ко-многим
    one_to_many = [
        (s.fio, s.score, g.name)
        for g in groups
        for s in students
        if s.group_id == g.id
    ]
    # 2. Соединение данных многие-ко-многим
    many_to_many_temp = [
        (g.name, sg.group_id, sg.student_id)
        for g in groups
        for sg in students_groups
        if g.id == sg.group_id
    ]
    many_to_many = [
        (s.fio, s.score, group_name)
        for group_name, group_id, student_id in many_to_many_temp
        for s in students
        if s.id == student_id
    ]
    # Задание 1 (Вариант В)
    # Вывести список всех сотрудников (студентов), у которых фамилия начинается с буквы «А», и названия их отделов (групп)
    print('Задание 1')
    res_v1 = [(s_fio, g_name) for s_fio, s_score, g_name in one_to_many if s_fio.startswith('А')]
    print(res_v1)
  # Задание 2 (Вариант В)
  # Вывести список отделов (групп) с минимальной зарплатой (баллом) сотрудников (студентов) в каждом отделе,
  # отсортированный по минимальной зарплате (баллу).
  print('\nЗадание 2')
  res_v2_unsorted = []
  for g in groups:
      g_students = [item for item in one_to_many if item[2] == g.name]
      if len(g_students) > 0:
          scores = [st[1] for st in g_students]
          min_score = min(scores)
          res_v2_unsorted.append((g.name, min_score))
  # Сортировка по минимальному баллу по возрастанию через lambda
  res_v2 = sorted(res_v2_unsorted, key=lambda x _T: x[1])
  print(res_v2)
  # Задание 3 (Вариант В)
  # Вывести список всех связанных сотрудников (студентов) и отделов (групп),
  # отсортированный по сотрудникам (студентам), сортировка по отделам (группам) произвольная.
  print('\nЗадание 3')
  # many_to_many содержит кортежи: (ФИО, балл, название_группы)
  # Сортируем по первому элементу (ФИО студента) через lambda
  res_v3 = sorted(many_to_many, key=lambda x _T: x[0])
  print(res_v3)
```

Результат:

```
Задание 1
[('Аристархов', 'ИУ5-21Б'), ('Андреев', 'ИУ5-21Б')]

Задание 2
[('ИУ6-22Б', 3.8), ('ИУ7-23Б', 4.0), ('ИУ5-21Б', 4.5)]

Задание 3
[('Андреев', 4.9, 'ИУ5-21Б'), ('Аристархов', 4.5, 'ИУ5-21Б'), ('Аристархов', 4.5, 'ИУ6-22Б'), ('Иванов', 4.0, 'ИУ7-23Б'), ('Петров', 3.8, 'ИУ6-22Б')]
```